Software Engineering for Software as a Service

Armando Fox, David Patterson
Spring 2012

Home

Quizzes

Assignments

Video Lectures

Forums

Course Outline

Course Outline

Week One:

1. Engineering SW is Different from HW (§1.1-§1.2)
2. Development Processes: Waterfall vs. Agile (§1.3)
3. Assurance (§1.4)
4. Productivity (§1.5)
5. Software as a Service (§1.6)
6. Service Oriented Architecture (§1.7)
7. Cloud Computing (§1.8)
8. Client-Server Architecture, HTTP, URIs, Cookies (§2.1-2.2)
9. HTML & CSS, XML & XPath (§2.2-2.3)
10. 3-tier shared-nothing architecture, horizontal scaling (§2.4)

Week Two:

1. Three pillars of Ruby (§3.1)
2. Everything is an object, and every operation is a method call (§3.2–3.3)
3. OOP in Ruby (§3.4)
4. Reflection and metaprogramming (§3.5)
5. Functional idioms and iterators (§3.6)
6. Model-View-Controller Design Pattern (§2.5)
7. Models: ActiveRecord and CRUD (§2.6)
8. Routes, Controllers and REST (§2.7)
9. Template Views (§2.8)

Homework 1: In this homework you will do some simple programming exercises to get familiar with the Ruby language. We will provide detailed automatic grading of your code.

Week Three:

1. Duck typing and mix-ins (§3.7)
2. Blocks and Yield (§3.8)
3. Rails Basics: Routes & REST (§3.9)
4. Databases and Migrations (§3.10)
5. ActiveRecord Basics (§3.11)
6. Controllers and Views (§3.12)
7. When things go wrong: Debugging (§3.13)
8. Forms (§3.14)
9. Redirection, the Flash and the Session (§3.15)
10. Finishing CRUD (§3.16)

Homework 2: In this homework you will clone a GitHub repo containing an existing simple Rails app, add a feature to the app, and deploy the result publicly on the Heroku platform. We will run live integration tests against your deployed version.

Week Four:

1. Introduction to Behavior-Driven Design and User Stories (§4.1)
2. SMART User Stories (§4.2)
3. Introducing and Running Cucumber and Capybara (§4.3-§4.4)
4. Lo-Fi UI Sketches and Storyboards (§4.5)
5. Enhancing Rotten Potatoes Again (§4.6)
6. Explicit vs. Implicit and Imperative vs. Declarative Scenarios (§4.7)
7. Fallacies & Pitfalls, BDD Pros & Cons (§4.8-§4.9)

Homework 3: In this homework you will create user stories to describe a feature of a SaaS app, use the Cucumber tool to turn those stories into executable acceptance tests, and run the tests against your SaaS app.

Week Five:

1. Testing Overview (§5.1)
2. FIRST, TDD and Getting Started with RSpec (§5.2)
3. The TDD Cycle: Red-Green-Refactor (§5.3)
4. More Controller Specs and Refactoring (§5.4)
5. Fixtures and Factories (§5.5)
6. TDD for the Model & Stubbing the Internet (§5.6-§5.7)
7. Coverage, Unit vs. Integration Tests, Other Testing Concepts, and Perspectives (§5.8-§5.11)

Homework 4: In this homework you will use a combination of Behavior-Driven Design (BDD) and Test-Driven Development (TDD) with the Cucumber and RSpec tools to add a new feature to a SaaS app, and deploy the resulting

app on Heroku.

Homework 5: In this homework you will add more advanced features to a SaaS app, including associations among different entity types, third-party authentication such as Facebook Connect, and some JavaScript/AJAX functionality (exact details of HW 5 are still TBD).

Created Mon 20 Feb 2012 7:50:25 PM PST
Last Modified Wed 22 Feb 2012 3:38:31 AM PST